

Proof-Driven Python Code Generation via Agentic PlusCal Invariant Co-Synthesis

Harry Wang, Eric Shang Nan Chen

Description

Current LLM code generation workflows produce code first and attempt verification or debugging after the generation. This project inverts that by building an autonomous agentic system that generates the correctness proof and the code side-by-side, which makes sure that no code is produced without a corresponding formal guarantee.

Our framework will take in a natural language requirement, and the agent co-generates a PlusCal specification paired with a candidate inductive invariant. It then validates the invariant against the three proof obligations:

- **Initiation:** $\text{Init} \Rightarrow \text{Inv}$
- **Consecution:** $\text{Inv} \wedge \text{Next} \Rightarrow \text{Inv}'$
- **Property Implication:** $\text{Inv} \Rightarrow \text{Property}$

If any check fails, the counterexample is fed through a feedback loop for repair. When it is verified an agent refines the PlusCal into executable Python.

Ideas:

1. **Auto-Formalization:** The LLM agent translates natural language prompt into a PlusCal algorithm paired with a candidate inductive invariant, which the PlusCal translator (`pca1`) compiles into a TLA⁺ Next formula. We can constrain the LLM to produce structured PlusCal + invariant pairs rather than raw TLA⁺, which can reduce syntax errors according to results from this paper (<https://arxiv.org/html/2512.09758>)
2. **Verification:** The orchestration layer runs the generated spec through TLC for model checking. If verification fails, the specific counterexample trace is fed back to the agent as structured tool feedback. The agent will diagnose which of the three checks failed and repairs either the invariant or the spec. This cycle repeats until all checks pass.
3. **Code Improvement:** Another agent will take the PlusCal spec and make it Python modules. We will put in runtime assertions from the invariant conditions.

Tool Stack

- **LLM backbone:** Claude or GPT-4o via API with function calling
- **Agent tools:** `run_tlc_check(spec)`, `translate_pluscal(spec)`, structured error parser
- **Verification engine:** TLC model checker (command-line invocation via Python subprocess)
- **PlusCal support:** Official `pca1` translator, `jessemyers/pluscal` Python AST builder
- **Few-shot data:** tlplus/Examples corpus, TLAiBench benchmark suite